

2 - 数据的类型及机器级表示

作者：Klein White

时间：2025-04-21

概要：

机器级数据分两大类：

- 数值数据：无符号整数、带符号整数、浮点数（实数）
- 非数值数据：逻辑数（包括位串）、西文字符和汉字

计算机内部所有信息都用二进制（即：0和1）进行编码

用二进制编码的原因：

- 制造二个稳定态的物理器件容易
- 二进制编码、计数、运算规则简单
- 正好与逻辑命题对应，便于逻辑运算，并可方便地用逻辑电路实现算术运算

数值数据表示的三个要素：

- 1、进位计数制：即该数值是使用二进制、八进制、十进制。。
- 2、定、浮点表示：解决小数点问题
- 3、如何用二进制编码：原码、补码、反码等（解决数值的正负号问题）

进位计数制度

非十进制转为十进制：按权相加法

十进制转为非十进制：除基取余法

十进制小数转为非十进制小数：乘基取整法

二进制编码

1、原码

Decimal	Binary	Decimal	Binary
0	0000	-0	1000
1	0001	-1	1001
2	0010	-2	1010
3	0011	-3	1011
4	0100	-4	1100
5	0101	-5	1101
6	0110	-6	1110
7	0111	-7	1111

◆ 容易理解, 但是:

- ✓ 0 的表示不唯一, 故不利于程序员编程
- ✓ 加、减运算方式不统一
- ✓ 需额外对符号位进行处理, 故不利于硬件设计
- ✓ 特别当 $a < b$ 时, 实现 $a - b$ 比较困难

2、补码: 模 n 同余

比如: 模12系统中, 8和-4互为对方的补码

- 一个负数的补码等于模减该负数的绝对值
- 数字0的表示是唯一的
- 比原码和反码多表示一个最小负数
- 加法与减法的统一: 对于某一确定的模, 某数减去小于模的另一数, 总可以用该数加上另一数负数的补码来代替
- 一个负数的补码等于对应正数补码的“各位取反、末位加1”, 比如-1就是1的原码取反加1
- 补码的第一位即为符号位, 1表示负数, 0表示正数

计算 $0111\ 1111 - 0100\ 0000 = ?$

$$\begin{aligned} 0111\ 1111 - 0100\ 0000 &= 0111\ 1111 + (2^8 - 0100\ 0000) \\ &= 0111\ 1111 + 1100\ 0000 = \boxed{1}0011\ 1111 \pmod{2^8} \\ &= 0011\ 1111 \end{aligned}$$


只留余数, “1” 被丢弃

变形补码：原本的补码只有一个符号位，变形补码有两个符号位，用于存放可溢出的中间结果。
如果符号位的两位不同，则说明计算溢出了

- ◆ 正数：符号位（sign bit）为0，数值部分不变
- ◆ 负数：符号位为1，数值部分“各位取反，末位加1”

变形（模4）补码：双符号，用于存放可溢出的中间结果。

	Decimal	补码	变形补码	Decimal	Bitwise Inverse	补码	变形补码
+0和-0 表示唯一	0	0000	00000	-0	1111	0000	00000
	1	0001	00001	-1	1110	1111	11111
	2	0010	00010	-2	1101	1110	11110
	3	0011	00011	-3	1100	1101	11101
	4	0100	00100	-4	1011	1100	11100
	5	0101	00101	-5	1010	1011	11011
	6	0110	00110	-6	1001	1010	11010
	7	0111	00111	-7	1000	1001	11001
	8	1000	01000	-8	0111	1000	11000

值太大，用4位补码无法表示，故“溢出”！但用变形补码可保留符号位和最高数值位。

3、移码

Excess (biased) notion- 移码表示

° 什么是“excess (biased) notation-移码表示”？

将每一个数值加上一个偏置常数 (Excess / bias)

° 一般来说，当编码位数为 n 时，bias取 2^{n-1}

Ex. $n=4$: $E_{\text{biased}} = E + 2^3$ (bias = $2^3 = 1000B$)

-8 (+8) ~ 0000B

-7 (+8) ~ 0001B

...

0 (+8) ~ 1000B

...

+7 (+8) ~ 1111B

0的移码表示唯一

移码和补码仅第一位不同

移码主要用来表示

浮点数阶码！

° 为什么要用移码来表示指数（阶码）？

便于浮点数加减运算时的对阶操作（比较大小）

例： $1.01 \times 2^{-1} + 1.11 \times 2^3$

补码： 111 < 011 ?
(-1) (3)

简化比较

$1.01 \times 2^{-1+4} + 1.11 \times 2^{3+4}$

移码： 011 < 111
(3) (7)

4、反码

就是每一位都取反的

为什么用补码来表示带符号整数

- 补码运算系统是模运算系统，加、减运算统一
- 数0的表示唯一，方便使用
- 比原码和反码多表示一个最小负数
- 与移码相比，其符号位和真值的符号对应关系清楚

无符号整数

无符号整数的编码里面没有符号位，这样它可表示的正整数的范围会更大。一般在运算过程中不出现负数的时候使用。无符号数溢出的时候不会产生“溢出异常”

LSB：最低有效位

MSB：最高有效位

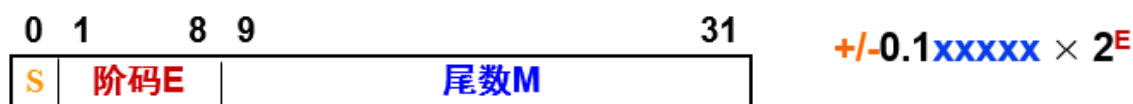
机器中字的位排列顺序有两种方式：

1、高到低位从左到右（常见的）--> LSB在右边，MSB在左边 2、高到低位从右到左

浮点数表示

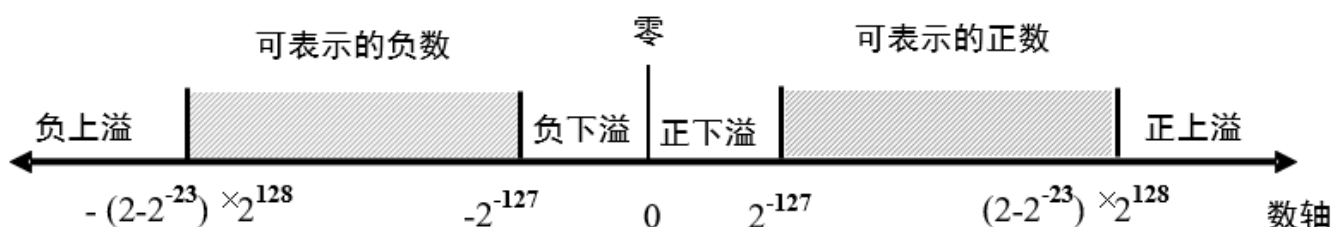
浮点数(Floating Point)的表示范围

例：画出下述32位浮点数格式的规格化数的表示范围。



第0位数符S；第1~8位为8位移码表示阶码E（偏置常数为127）；第9~31位为23位二进制原码小数表示的尾数M。规格化尾数的小数点后第一位总是1，故规定第一位默认的“1”不明显表示出来。这样可用23个数位表示24位尾数。

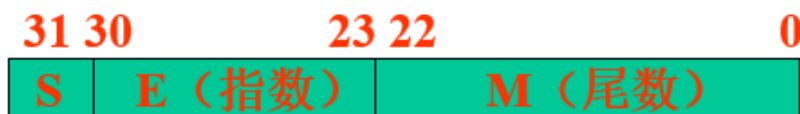
最大正数： $1.1...1 \times 2^{11...1} = (2-2^{-23}) \times 2^{128}$ 最小正数： $1.0...0 \times 2^{00...0} = 1.0 \times 2^{-127}$
因为原码是对称的，所以其表示范围关于原点对称。



机器0：尾数为0 或 落在下溢区中的数

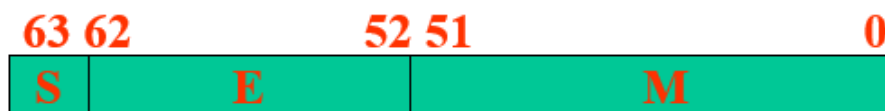
浮点数范围比定点数大，但数的个数没变多，故数之间更稀疏，且不均匀

(1) 单精度浮点数，如下：



$$X = (-1)^S * (1.M) * 2^{E-127}$$

(2) 双精度浮点数，如下：



$$X = (-1)^S * (1.M) * 2^{E-1023}$$

由于在规格化二进制浮点数中，小数点前面的一位数总是1，故可将这个1省略，不出现在浮点数的格式中，我们称其为隐藏位（hidden bit）。

-12.75

1. Denormalize: -12.75

2. Convert integer part:

$$12 = 8 + 4 = 1100_2$$

3. Convert fractional part:

$$.75 = .5 + .25 = .11_2$$

4. Put parts together and normalize:

$$1100.11 = 1.10011 \times 2^3$$

5. Convert exponent: $127 + 3 = 128 + 2 = 1000\ 0010_2$

11000 0010	100 1100 0000 0000 0000 0000
------------	------------------------------

The Hex rep. is **C14C0000H**

需要注意，浮点数表示0的有两种方式，两种方式都是合法的：

+0: 0 00000000 000000000000000000000000000000

-0: 1 00000000 000000000000000000000000000000

NaN: Not a Number

Operations

$$\text{sqrt}(-4.0) = \text{NaN}$$

$$\text{op}(\text{NaN}, x) = \text{NaN}$$

$$+\infty - (+\infty) = \text{NaN}$$

etc.

$$0/0 = \text{NaN}$$

$$+\infty + (-\infty) = \text{NaN}$$

$$\infty/\infty = \text{NaN}$$

西文字符：如：abcABC和各自符号，其使用 ASCII 码作为内码来表示

汉字字符：国标码（国标交换码）可在汉字国标码的基础上产生汉字机内码。至少需2个字节才

能表示一个汉字内码，这是由汉字的字数决定的

数据的基本单位

比特 (bit) 是计算机中处理、存储、传输信息的最小单位 (0 or 1)

二进制信息的计量单位是“字节”，现代计算机中，存储器是**按照字节编址**，字节是最小的可寻址单位

◆ 存储二进制信息时的度量单位要比字节或字大得多

◆ 主存容量经常使用的单位，如：

- “千字节” (KB), $1\text{KB}=2^{10}\text{字节}=1024\text{B}$
- “兆字节” (MB), $1\text{MB}=2^{20}\text{字节}=1024\text{KB}$
- “千兆字节” (GB), $1\text{GB}=2^{30}\text{字节}=1024\text{MB}$
- “兆兆字节” (TB), $1\text{TB}=2^{40}\text{字节}=1024\text{GB}$

◆ 主频和带宽使用的单位，如：

- “千比特/秒” (kb/s), $1\text{kbps}=10^3\text{ b/s}=1000\text{ bps}$
- “兆比特/秒” (Mb/s), $1\text{Mbps}=10^6\text{ b/s}=1000\text{ kbps}$
- “千兆比特/秒” (Gb/s), $1\text{Gbps}=10^9\text{ b/s}=1000\text{ Mbps}$
- “兆兆比特/秒” (Tb/s), $1\text{Tbps}=10^{12}\text{ b/s}=1000\text{ Gbps}$

数据的存储和排列顺序

需要考虑的两个问题：

- 1、如何根据一个地址取到一个32位的字
- 2、一个字能否存放在任何地址边界

例如，若 `int i = -65535`，存放在内存100号单元（即占100#~103#），则用“取数”指令访问100号单元取出 `i` 时，必须清楚 `i` 的4个字节是如何存放的。

Word:					little endian word 100#
	FF	FF	00	01	
	103	102	101	100	
	msb			lsb	
	100	101	102	103	big endian word 100#

大端方式（**Big Endian**）：**MSB**所在的地址是数的地址

e.g. IBM 360/370, Motorola 68k, MIPS, Sparc, HP PA

小端方式（**Little Endian**）：**LSB**所在的地址是数的地址

e.g. Intel 80x86, DEC VAX

是以 LSB 作为数据的地址，还是以MSB作为数据的地址？ --> 数据的存储问题

Memory layout of a number 00ABCDEFH located in 1000

In Big Endian:	→	00	1000
		AB	1001
		CD	1002
		EF	1003
In Little Endian:	→	00	1003
		AB	1002
		CD	1001
		EF	1000

数据在存储中的排列情况：

要求数据的地址是相应的边界地址

◆ 各种不同长度的数据存放时，有两种处理方式：

- 按边界对齐（假定存储字的宽度为32位，按字节编址）

- 字地址：4的倍数（低两位为0）

- 半字地址：2的倍数（低位为0）

- 字节地址：任意

每4个字节可同时读写

- 不按边界对齐

坏处：可能会增加访存次数！

（学了第四章存储器组织后会更明白！）

如：int i, short k, double x, char c, short j,.....

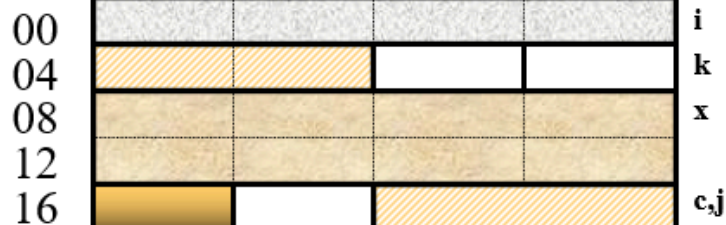
存储器按字节
编址

每次只能读写
某个字地址开
始的4个单元中
连续的1个、2
个、3个或4个
字节

按边界对齐

x: 2个周期

j: 1个周期



则：&i=0; &k=4; &x=8; &c=16; &j=18;.....

字节0 字节1 字节2 字节3

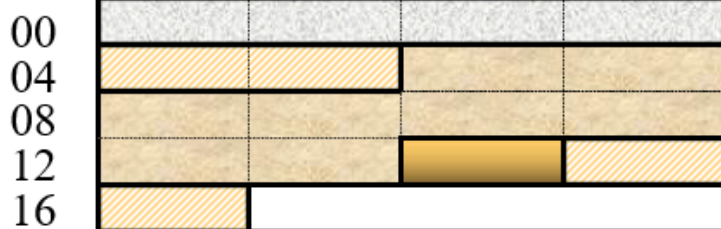
虽节省空间，但
增加访存次数！

性能更重要，浪
费一点存储空间
没关系！

边界不对齐

x: 3个周期

j: 2个周期



则：&i=0; &k=4; &x=6; &c=14; &j=15;.....